

Formalized Hopfield Networks and Boltzmann Machines

Matteo Cipollina[†] ✉

Università Cattolica del Sacro Cuore, Milan, Italy

Michail Karatarakis[†] ✉ 🏠 

Institute for Computing and Information Sciences, Radboud University, Nijmegen, The Netherlands

Freek Wiedijk ✉ 🏠 

Institute for Computing and Information Sciences, Radboud University, Nijmegen, The Netherlands

Abstract

Neural networks are widely used, yet their analysis and verification remain challenging. In this work, we present a Lean 4 formalization of neural networks, covering both deterministic and stochastic models. We first formalize Hopfield networks, recurrent networks that store patterns as stable states. We prove convergence and the correctness of Hebbian learning, a training rule that updates network parameters to encode patterns. We then consider stochastic networks, where updates are probabilistic and convergence is to a stationary distribution. As a canonical example, we formalize the dynamics of Boltzmann machines and prove their ergodicity, showing convergence to a unique stationary distribution using a new formalization of the Perron-Frobenius theorem.

2012 ACM Subject Classification Theory of computation → Logic and verification; Theory of computation → Type theory

Keywords and phrases formal math, type theory, Lean 4, mathlib, neural networks, Hopfield networks, PhysLean, Boltzmann Machines

Acknowledgements The authors would like to thank Britt Anderson for proposing this project, and Niels van der Weide for his valuable feedback on previous versions of this manuscript.

1 Introduction

Artificial neural networks are information processing systems inspired by biological brains. The study of artificial neural networks is highly interdisciplinary, drawing from fields such as computer science, neuroscience, mathematics, and engineering. These networks are capable of learning from experience and extracting patterns from complex and seemingly unrelated data. This ability to learn makes neural networks a powerful tool in artificial intelligence, but it also raises the need for formal verification – not only of their outputs, but also of the mathematical foundations underlying these models.

In October 2024, John Hopfield and Geoffrey Hinton were awarded the Nobel Prize in Physics for their work in machine learning, particularly in the development of artificial neural networks [1]. One of Hopfield’s most influential achievements is the Hopfield network [27], introduced in 1982. Hopfield networks model associative memory, storing and recalling patterns from partial or noisy input, and were originally inspired by the Ising model of magnetism [28]. They are valuable across fields such as physics, psychology, neuroscience, and machine learning, due to their connections to statistical mechanics, recurrent neural networks, and cognitive psychology. A key feature is their convergence property, where the network reaches a stable state corresponding to a stored pattern.

[†] Equal contribution

Building on Hopfield networks, Boltzmann machines were introduced by Ackley, Hinton and Sejnowski [4] in 1985 as a stochastic generalization. In Boltzmann machines, neuron updates are probabilistic rather than deterministic, allowing the network to model distributions over binary states instead of converging to a single stable state. This stochasticity makes Boltzmann machines a powerful tool for probabilistic modeling and unsupervised learning, but also introduces additional challenges for convergence and formal verification. By formalizing both deterministic Hopfield networks and stochastic Boltzmann machines, we aim to provide a rigorous foundation for reasoning about both types of networks and their learning dynamics.

In this paper, we present a Lean 4 formalization of Hopfield networks and Boltzmann machines, inspired by a suggestion from Britt Anderson in the Lean Zulip chat [7]. To our knowledge, this is the first formalization of Hopfield networks and the dynamics of Boltzmann machines.

Contributions

- We formalize the notion of a general neural network and its computational behavior (§2).
 - We formalize discrete symmetric Hopfield networks, prove their convergence, and implement the Hebbian learning algorithm (§3).
 - We formalize probabilistic concepts – including reversibility, invariance of Markov kernels, Gibbs sampling, and the Perron–Frobenius theorem – with applications to ergodicity (§4).
 - We formalize Boltzmann machines and prove their convergence to the Boltzmann distribution (§4.5).
- We detail the key design decisions of our formalization (§5), survey related work (§6), and outline directions for future research (§7). A repository corresponding to this paper is publicly available[†].

Implementation comments

An important goal of this project is to contribute to the Lean ecosystem. Our development contains 15,342 lines of code and leverages `mathlib`'s [34] probability and graph theory APIs, adhering to its philosophy of formalizing results in maximal generality rather than on an ad hoc basis. The mathematical infrastructure we developed has either been integrated into `mathlib` or is currently under review. The remaining formalization, covering neural network theory, has been submitted as an open pull request to `PhysLean` [47], a Lean 4 physics library built on `mathlib`, and is currently under review.

2 General Neural Networks

Neural networks can be represented as directed graphs: if the graph is acyclic, the network is feedforward; if it contains cycles, it is recurrent. The edges of the graph are labeled with *weights*, which are determined when training the network (or in the case of the Hebbian learning rule, just computed). The vertices of the graph, the *neurons*, have *activations* which are updated when executing the network. At that time the weights (and other parameters) of the network generally stay fixed. We begin by defining the structure and operation of

[†] Anonymized GitHub link: [link to be provided after review].

(artificial) neural networks, followed by examples with corresponding Lean computations, guided by Chapter 4 of [30]. Our work builds on basic graph theory results from `mathlib`.

2.1 Definitions

An (artificial) neural network is a directed graph $G = (U, C)$, where neurons $u \in U$ are connected by directed edges $c \in C$ (connections). The neuron set is partitioned as $U = U_{\text{in}} \cup U_{\text{out}} \cup U_{\text{hidden}}$, with $U_{\text{in}}, U_{\text{out}} \neq \emptyset$ and $U_{\text{hidden}} \cap (U_{\text{in}} \cup U_{\text{out}}) = \emptyset$. Each connection $(v, u) \in C$ has a weight w_{uv} , and each neuron u has real-valued quantities: network input net_u , activation act_u , and output out_u . Input neurons $u \in U_{\text{in}}$ also have a fourth quantity, the external input ext_u . The predecessors and successors of a vertex u in a directed graph $G = (U, C)$ are defined as $\text{pred}(u) = \{v \in V \mid (v, u) \in C\}$ and $\text{succ}(u) = \{v \in V \mid (u, v) \in C\}$ respectively. Each neuron u is associated with the following functions:

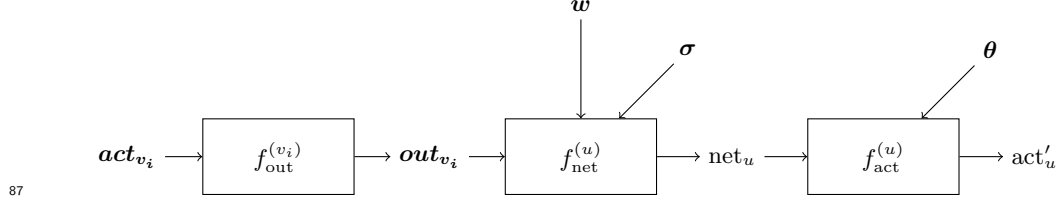
$$f_{\text{net}}^{(u)} : \mathbb{R}^{2|\text{pred}(u)| + \kappa_1(u)} \rightarrow \mathbb{R}, \quad f_{\text{act}}^{(u)} : \mathbb{R}^{1 + \kappa_2(u)} \rightarrow \mathbb{R}, \quad f_{\text{out}}^{(u)} : \mathbb{R} \rightarrow \mathbb{R}.$$

These functions compute net_u , act_u , and out_u , where $\kappa_1(u)$ and $\kappa_2(u)$ count the number of parameters of those functions, which can depend on the neurons. Specifically, the new activation act'_u of a neuron u is computed as follows:

$$\text{act}'_u = f_{\text{act}}^{(u)}(f_{\text{net}}^{(u)}(w_{uv_1}, \dots, w_{uv_{\text{pred}(u)}}), f_{\text{out}}^{(v_1)}(\text{act}_{v_1}), \dots, f_{\text{out}}^{(v_{\text{pred}(u)}})(\text{act}_{v_{\text{pred}(u)}}), \boldsymbol{\sigma}^{(u)}), \boldsymbol{\theta}^{(u)})$$

where $\boldsymbol{\sigma}^{(u)} = (\sigma_1^{(u)}, \dots, \sigma_{\kappa_1(u)}^{(u)})$ and $\boldsymbol{\theta} = (\theta_1^{(u)}, \dots, \theta_{\kappa_2(u)}^{(u)})$ are the input parameter vectors.

The following diagram shows the structure of a generalized neuron where $\mathbf{act}_{v_i} = (\text{act}_{v_1}, \dots, \text{act}_{v_{|U|}})$ and $\mathbf{out}_{v_i} = (\text{out}_{v_1}, \dots, \text{out}_{v_{|U|}})$.



The term *generalized* neuron is used to emphasize that the neuron is not assumed to follow a specific neural network model, but is parameterized by arbitrary input, activation, and output functions, as well as an arbitrary graph-based connectivity structure.

In the index of a weight w_{uv} , the neuron receiving the connection is listed first, following the ‘row first, then column’ convention of the adjacency matrix. This structure groups weights leading to a neuron in the same row, with $n = |U|$:

$$W = \begin{pmatrix} w_{u_1 u_1} & w_{u_1 u_2} & \cdots & w_{u_1 u_n} \\ w_{u_2 u_1} & w_{u_2 u_2} & \cdots & w_{u_2 u_n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{u_n u_1} & w_{u_n u_2} & \cdots & w_{u_n u_n} \end{pmatrix}$$

In our implementation, we make some changes to the textbook definition above. To ensure flexibility and generality across network structures, we use arbitrary types (`R : Type`) for weights and activations, with `[Zero R]` indicating the existence of a zero element, instead of the real numbers. Similarly, we use (`U : Type`) for the neurons.

Also, in the textbook that we are following, function arguments and summations for a neuron u are generally indexed running over the set $\text{pred}(u)$. In Lean that could correspond

to passing around adjacency proof objects everywhere. For example the weight matrix could have been written as $(w : \forall u v : U, \text{Adj } v u \rightarrow R)$.

Similarly, the type of $f_{\text{net}}^{(u)} : \mathbb{R}^{2|\text{pred}(u)|+\kappa_1(u)} \rightarrow \mathbb{R}$ in Lean would have been :

```
103 (fnet : ∀ u : U, (∀ v : U, Adj v u → R) → (∀ v : U, Adj v u → R) →
104   Vector R (κ1 u) → R)
```

This then explicitly needs tracking adjacency relations, complicating everything. To simplify our formalization, we decided not to do this. Instead we use the convention that all weights for neurons that are not adjacent are zero. Therefore we use a matrix $(w : \text{Matrix } U U R)$ directly and the function formalizing $f_{\text{net}}^{(u)}$ gets type

```
112 (fnet : ∀ u : U, (U → R) → (U → R) → Vector R (κ1 u) → R)
```

However, this may be less efficient for sparse networks that require precise connection control. Also, the functions might be harder to implement with this convention. A solution for this would be to have the adjacency information as extra arguments of type $(U \rightarrow \text{Bool})$, but because our applications did not need this, it is not in our formalization.

Finally, we separate a neural network into its architecture, defined by its graph structure, and parameters, controlling dynamics like input processing. This separation allows flexibility, enabling different functions, weights, or learning rules without redefining the architecture. Occasionally, in order to save space in the Lean code snippets, we replace the full proofs propositions with ‘...’.

Our formalization initially modeled neural networks using `mathlib`’s `Digraph` structure, which defines a graph as a binary relation on vertices:

```
126 structure Digraph (V : Type*) where Adj : V -> V -> Prop
```

While this propositional representation (`Prop`) suffices for determining simple connectivity, it is insufficient for the thermodynamic proofs presented in Section ??.

Specifically, the trace formula in the Perron-Frobenius theorem requires a rigorous algebra of paths, where distinct paths between the same endpoints must be distinguishable to count cycles (a requirement for computing partition functions).

To support this, we refactored our definition to use `mathlib`’s `Quiver` typeclass. As defined in the library, `Quiver` is universe-polymorphic over the sort of the morphism type:

```
136 class Quiver (V : Type u) where Hom : V -> V -> Sort v
```

This polymorphism provides a strict generalization of the `Digraph` structure. As noted in the `mathlib` source documentation, a `Digraph V` is structurally equivalent to a `Quiver.{0} V`, where the morphism sort is instantiated to `Prop`. By adopting `Quiver` as our foundation, we allow our networks to be instantiated in `Type` (i.e., `Quiver.{v+1}`), enabling the representation of multigraphs with distinct synaptic connections.

This design unifies the verification landscape: standard sequential architectures (which prohibit multiple edges) are naturally embedded as `Quiver.{0}` instances (Section ??), while the recurrent energy-based models required for our ergodicity results utilize the full expressive power of `Quiver.{1}`.

The following snippet captures the essential structure of a neural network: the partition of neurons $U_{\text{in}}, U_{\text{out}}, U_{\text{hidden}}$, the parameter dimensions $\kappa_1(u)$ and $\kappa_2(u)$, and the functions $f_{\text{net}}^{(u)}, f_{\text{act}}^{(u)}, f_{\text{out}}^{(u)}$ computing net input, activation, and output respectively for each neuron $u \in U$. For brevity, we omit well-formedness conditions, the aforementioned adjacency relations, and predicates on activations and weights, which are fully specified in the supplemented code.

```
153 universe u
```

```

155 2 structure NeuralNetwork (R U : Type u) [Zero R] extends Quiver.{u+1} U
156   where
157 3   (Ui Uo Uh : Set U)
158 4   ( $\kappa1 \kappa2 : U \rightarrow \mathbb{N}$ )
159 5   (fnet :  $\forall u : U, (U \rightarrow R) \rightarrow (U \rightarrow R) \rightarrow$ 
160 6     Vector R ( $\kappa1 u$ )  $\rightarrow R$ )
161 7   (fact :  $\forall u : U, R \rightarrow$  Vector R ( $\kappa2 u$ )  $\rightarrow R$ )
162 8   (fout :  $\forall u : U, R \rightarrow R$ )
163 9   ...

```

Similarly, the structure `Params` specifies the parameters of the neural network `NN`. Concretely, it consists of the weight matrix w , the input vector σ_u for each neuron $u \in U$, and the threshold vector θ_u for each neuron $u \in U$, as defined in the Lean code snippet below:

```

169 1 structure Params (NN : NeuralNetwork R U) where
170 2   (w : Matrix U U R)
171 3   ( $\sigma : \forall u : U, \text{Vector } R \text{ (NN.}\kappa1 \text{ } u)$ )
172 4   ( $\theta : \forall u : U, \text{Vector } R \text{ (NN.}\kappa2 \text{ } u)$ )
173 5   ...

```

The full formalization additionally includes proofs that unconnected neurons have zero weights and that the weight matrix satisfies the required properties.

A state of the network is simply a snapshot of all neuron activations at a given discrete time step. Formally, this means that a state assigns a real number to each neuron, subject to basic admissibility conditions (for example, that activations lie in a prescribed range). No notion of time is built into the state itself: a state should be thought of as ‘the configuration of the network at one instant’. The network’s state is represented by the `State` structure, which includes neuron activations and a proof that they satisfy the required properties.

```

184 1 structure State (NN : NeuralNetwork R U) where
185 2   act : U  $\rightarrow$  R
186 3   ...

```

To describe a neural network, we specify how each neuron computes its output from its inputs and how computations across neurons are organized, including external input processing and neuron update order.

We introduce variables for the neural network parameters and the state structure:

```

192 1 variable {NN : NeuralNetwork R U} (s : NN.State) (w $\theta$  : Params NN)

```

Then functions such as `s.fout` and `s.fnet` are defined in terms of the activations stored in a given `State`. In this sense, they play the role of the network’s `NN.fout` and `NN.fnet` for the current state. The function `s.Up w $\theta$  u` produces a new state where `u`’s activation is updated using the network’s activation function `NN.fact`, while all other activations remain unchanged.

Two-state networks

We defined neural networks using $\{0,1\}$ activations. Hopfield networks use $\{-1,+1\}$, while Boltzmann machines also use $\{0,1\}$; these choices shape both the computations and the structure of proofs. To unify architectures, we introduce the `TwoStateNeuralNetwork` typeclass, specifying two activation states, a threshold-based update function, and an ordering to a numeric type. This allows proofs to apply uniformly across representations; $\{-1,+1\}$ (`SymmetricBinary`), $\{0,1\}$ (`ZeroOne`), or custom types such as `Signum`.

2.2 Sequential Architectures

Most verification literature models feedforward networks as inductive lists of layers [2, 15]. A *sequential architecture* is a directed graph $G = (U, C)$, where the vertex set is partitioned into disjoint layers $\mathcal{L} = (L_0, \dots, L_k)$ such that $L_0 = U_{\text{in}}$ and $L_k = U_{\text{out}}$. The connection set is restricted to adjacent layers $C = \{(v, u) \mid u \in L_i \wedge v \in L_{i-1}\}$, implying $\text{pred}(u) = L_{i-1}$ for any $u \in L_i$ [cite: 167]. Each neuron u utilizes a parameter vector $\sigma^{(u)} = (\beta_u)$ for bias and $\theta^{(u)}$ for activation configuration. [cite_start]Specifically, the activation parameters $\theta^{(u)}$ are empty for standard models but contain a lookup index (id_u) for the Isabelle formalism [cite: 150, 153]. The generalized neuron functions for this architecture are defined as:

$$f_{\text{net}}^{(u)} = \sum_{v \in \text{pred}(u)} w_{uv} \text{out}_v + \sigma_1^{(u)}, \quad f_{\text{act}}^{(u)} = \begin{cases} \phi(\text{net}_u) & \text{Standard} \\ \text{lookup}(\theta_1^{(u)})(\text{net}_u) & \text{Isabelle} \end{cases}, \quad f_{\text{out}}^{(u)} = \text{act}_u.$$

Our **Quiver** formalism is general enough to encompass standard sequential architectures while accommodating arbitrary connectivity. Within our framework, these sequential models emerge as a specialized subclass: they correspond to stratified quivers where the neuron set is partitioned into disjoint layers $U = \bigcup_{l=0}^k L_l$, with morphisms restricted to connections between adjacent partitions $L_l \rightarrow L_{l+1}$.

We formalize this mathematical structure directly. The architecture tuple $\mathcal{A} = (w_0, \dots, w_L)$ corresponds to the **SequentialArch** structure, which bundles the layer widths with validity constraints (e.g., ensuring non-empty layers).

```

216 structure SequentialArch where
217   layer_widths : List Nat           -- Corresponds to the tuple A
218   activations   : List Activation
219   h_len_eq      : layer_widths.length = activations.length
220   h_min_layers  : layer_widths.length >= 2
221   h_pos_widths  : forall w in layer_widths, w > 0
222

```

■ **Listing 1** Definition of Sequential Architecture

We map the neuron set U to the type **SeqNeuron**. This type implements the partition $U = \bigcup L_l$ by indexing neurons as dependent pairs of a layer index l and a neuron index $j \in \{0, \dots, w_l - 1\}$.

```

227 structure SeqNeuron (arch : SequentialArch) where
228   layerIdx : Fin arch.layer_widths.length -- The layer index l
229   neuronIdx : Fin (arch.layer_widths.get layerIdx) -- The index within L_l
230

```

■ **Listing 2** The Neuron Space

The core contribution of this embedding is the rigorous translation of the implicit sequential connectivity C into our explicit **Quiver** morphisms. In the sequential model, the edge set is defined as $C = \{(v, u) \mid u \in L_i \wedge v \in L_{i-1}\}$. This corresponds to a boolean proposition: v is adjacent to u if $\text{layer}(v) + 1 = \text{layer}(u)$. To lift this into our quiver framework, we define the morphism type $\text{Hom } v \text{ } u$ using the **PLift** operator. This transforms the *proposition* of adjacency into a *type*, allowing the structural definition of the network's topology.

```

239 -- Propositional Adjacency matching C = { (v, u) | ... }
240 -- Checks if v is in layer i and u is in layer i+1
241 def SeqAdj (v u : SeqNeuron arch) : Prop :=
242   v.layerIdx.val + 1 = u.layerIdx.val
243

```

```

244
245 -- Type-level Morphisms (Quiver style)
246 def toNeuralNetwork (arch : SequentialArch) : NeuralNetwork R (SeqNeuron
247   arch) := {
248   -- We lift the Prop to Type to satisfy the Quiver interface
249   Hom := fun v u => PLift (SeqAdj v u),
250
251   -- Input/Output/Hidden partitions derived from layer indices
252   Ui := { u | u.layerIdx.val = 0 }, -- U_in
253   Uo := { u | u.layerIdx.val = arch.layer_widths.length - 1 }, -- U_out
254   ...
255 }
256

```

■ **Listing 3** Lifting Sequential Adjacency to Quiver Morphisms

257 This construction proves that the sequential model is a strict sub-category of our frame-
 258 work. The standard forward-pass semantics of a sequential network-typically defined
 259 as a fold over layers-are recovered by specifying a topological update order. We define
 260 **sequentialOrder** as a list comprehension that linearizes the vertices layer-by-layer: $\mathcal{O} =$
 261 $[(l, i) \mid 1 \leq l < |\mathbf{w}|, 0 \leq i < w_l]$. By passing this order to our general **workPhase** function, we
 262 prove that the general asynchronous update mechanics of our framework strictly simulate
 263 the synchronous layer-wise propagation of the sequential model. This allows us to apply our
 264 thermodynamic theorems (Section §4) directly to these restricted feedforward architectures,
 265 unifying the verification landscape.

266 2.3 Operation of neural networks

267 We now divide the neural network’s computation into two phases: the *input phase* and the
 268 *work phase*. In the input phase, input neurons in are set to the external inputs, while the
 269 others are initialized to 0. The output function is applied to these activations to ensure all
 270 neurons produce outputs.

271 The order in which neurons recompute their outputs is not fixed, though different schemes
 272 may suit various network types. Neurons may update simultaneously (synchronous update)
 273 using previous outputs, or one at a time (asynchronous update), incorporating recent outputs.

274 Feedforward networks typically follow a topological order to avoid redundant updates,
 275 while recurrent networks may exhibit behaviors like oscillations, depending on the update
 276 order. In practice, a stopping criterion based on updates, energy, or convergence measures
 277 can stabilize the network. We formalize this notion as the property **isStable**. A state s is
 278 considered stable if applying the update function $s.\text{Up}$ to any neuron u leaves its activation
 279 unchanged.

280 The function **seqStates** defines the evolution of a network over time as a sequence of
 281 states. Given a sequence of neurons **useq** to update at each step, the first state is the initial
 282 state s , and each subsequent state is obtained by updating the neuron specified by **useq** at
 283 that step. Formally, the $(n + 1)^{\text{th}}$ state is $(\text{seqStates useq } n).\text{Up } (\text{useq } n)$.

284 Finally, we combine all the updates using the **workPhase** function which takes an initial
 285 state **extu** and a list **uOrder** defining the update sequence. Using **List.foldl**, it updates
 286 each neuron in turn, leaving the others unchanged. The result is a new state **NN.State** in
 287 which every neuron in the list has been updated once.

Computations carried out by a general neural network

To illustrate the structure of a general neural network, we consider an example with three neurons ($U = \{u_1, u_2, u_3\}$) from (p. 42, [30]). Neurons u_1 and u_2 are input neurons ($U_{\text{in}} = \{u_1, u_2\}$) receiving external inputs ext_{u_1} and ext_{u_2} , respectively, while u_3 is the sole output neuron ($U_{\text{out}} = \{u_3\}$). This network has no hidden neurons ($U_{\text{hidden}} = \emptyset$). The network structure is described by the matrix

$$W = \begin{pmatrix} 0 & 0 & 4 \\ 1 & 0 & 0 \\ -2 & 3 & 0 \end{pmatrix}.$$

The network behavior is determined by the following functions for each neuron $u \in U$:

$$f_{\text{net}}^{(u)}(w_u, \text{in}_u) = \sum_{v \in U \setminus \{u\}} w_{uv} \text{out}_v, \quad f_{\text{act}}^{(u)}(\text{net}_u, \theta_u) = \begin{cases} 1 & \text{if } \text{net}_u \geq \theta_u, \\ 0 & \text{otherwise,} \end{cases} \quad f_{\text{out}}^{(u)}(\text{act}_u) = \text{act}_u.$$

We formalize this network as (`test : NeuralNetwork  $\mathbb{Q}$  (Fin 3)`), its adjacency matrix as (`test.M : Matrix (Fin 3) (Fin 3)  $\mathbb{Q}$`), input (`Ui := {0,1}`) and output neuron (`Uo := {2}`) sets as well as the activation and output functions above.

Now we proceed with the initialization phase. We initialize the input neurons u_1 and u_2 with $\text{ext}_{u_1} = 1$ and $\text{ext}_{u_2} = 0$, and the output neuron u_3 with an initial activation of 0 as (`test.extu : test.State`). Finally, the update of the neurons' outputs is done step by step during the work phase. For example, the network input to neuron u_3 is the weighted sum of the outputs of neurons u_1 and u_2 . If the sum is less than the threshold, the output is set to zero.

To simulate the network's work phase, we use the `#eval` command to evaluate the state after each update, with neurons updated in the order $u_3, u_1, u_2, u_3, u_1, u_2, \dots$.

```

307
308 1 #eval NeuralNetwork.State.workPhase wθ test.extu test.onlyUin
309   [2,0,1,2,0,1,2]
310 2 --The output of this code will be:
311 3 acts: ![0, 0, 0], outs: ![0, 0, 0], nets: ![0, 0, 0]
```

This indicates that after the work phase, all neurons are in a stable state with zero activations. If we choose a different update order, the network might not converge to a stable state. For example, if the neurons are updated in the order $u_3, u_2, u_1, u_3, u_2, u_1, u_3, \dots$, the outputs of the neurons oscillate indefinitely, and no stable state is reached:

```

317
318 1 #eval NeuralNetwork.State.workPhase wθ test.extu test.onlyUin
319   [2,1,0,2,1,0,2]
320 2 --This results in oscillating activations that match the initial ones.
321 3 acts: ![1, 0, 0], outs: ![1, 0, 0], nets: ![0, 1, -2]
```

The network's behavior depends on when the work phase is terminated. If terminated after step k with $(k-1) \bmod 6 < 3$, the output neuron u_3 will have an activation of 0. If terminated with $(k-1) \bmod 6 \geq 3$, the activation will be 1. In recurrent networks, this illustrates the potential for oscillatory behavior, where the output depends not only on the input but also on the number of updates. By contrast, Hopfield networks always converge to a stable state under asynchronous updates.

3 Hopfield networks

Before Hopfield's work, neural networks were primarily feedforward models, with unidirectional information flow. Hopfield networks introduced recurrent connections, enabling feedback between neurons, allowing them to model dynamic systems and handle more complex tasks. We follow the mathematical background in Chapter 8 of [30].

3.1 Description of a Hopfield Network

A Hopfield network is a neural network with graph $G = (U, C)$ as described in the previous section, that satisfies the following conditions: $U_{\text{hidden}} = \emptyset$, and $U_{\text{in}} = U_{\text{out}} = U$, $C = U \times U - \{(u, u) \mid u \in U\}$, i.e., no self-connections. The connection weights are symmetric, i.e., for all $u, v \in U$, we have $w_{uv} = w_{vu}$ when $u \neq v$. The activation of each neuron is either 1 or -1 depending on the input. There are no loops, meaning neurons don't receive their own output as input. Instead, each neuron u receives inputs from all other neurons, and in turn, all other neurons receive the output of neuron u . The neuron functions for the Hopfield network are defined for all $u \in U$ as:

$$f_{\text{net}}^{(u)}(w_u, \text{in}_u) = \sum_{v \in U \setminus \{u\}} w_{uv} \text{out}_v, \quad f_{\text{act}}^{(u)}(\text{net}_u, \theta_u) = \begin{cases} 1 & \text{if } \text{net}_u \geq \theta_u, \\ -1 & \text{otherwise,} \end{cases} \quad f_{\text{out}}^{(u)}(\text{act}_u) = \text{act}_u.$$

Assume a linearly ordered field and a finite, nonempty set of neurons U with decidable equality. The `HopfieldNetwork` structure defines a Hopfield network of type `(NeuralNetwork R U)`. Its adjacency relation, `(Adj u v := u ≠ v)`, connects each neuron to all others except itself, and the weight matrix is symmetric, enforced by `(pw w := w.IsSymm)`. The set of hidden neurons is empty, and all neurons serve as both inputs and outputs, represented by `Set.univ U`, the universal set of elements of type `U`. The network input (3.1), activation (3.1), and output (3.1) functions from the definition are represented by `HNfnet`, `HNfact`, and `HNfout`, respectively.

3.2 Hopfield networks converge

As shown in §2.3, updates can cause the network to oscillate between activation states. However asynchronous updates (where neurons are updated one at the time) in a Hopfield network always lead to a stable state, preventing oscillation.

► **Theorem 3.1** (Convergence Theorem for Hopfield networks (Theorem 8.1, [30])). *If the activations of the neurons of a Hopfield network are updated asynchronously, a stable state is reached in a finite number of steps. If the neurons are traversed in an arbitrary, but fixed cyclic fashion, at most $n \cdot 2^n$ steps (updates of individual neurons) are needed, where n is the number of neurons of the network.*

This theorem is proved by defining an energy function that assigns a real value to each state of the Hopfield network, which decreases or remains constant with each transition. When energy remains unchanged, we show that a state can never be revisited. Since there are only finitely many states, this ensures the network will eventually reach a stable state. The energy function of a Hopfield network with n neurons $u_1, \dots, u_n$ is

$$E = -\frac{1}{2} \sum_{\substack{u, v \in U \\ u \neq v}} w_{uv} \text{act}_u \text{act}_v + \sum_{u \in U} \theta_u \text{act}_u. \quad (1)$$

The factor $\frac{1}{2}$ compensates for the symmetry of the weights, as each term in the first sum appears twice.

We now introduce the variables for the Hopfield network states and the parameters :

```
360
361 variable {s : (HopfieldNetwork R U).State} (wθ : Params (HopfieldNetwork
362 1 R U))
363
364
```

The network energy defined in Equation (1) is formalized as (NeuralNetwork.State.E : R) and it is composed of two parts. The first part,

```
365
366 def NeuralNetwork.State.Ew := - 1/2 * (∑ u, (∑ v2 in {v2 | v2 ≠ u},
367 s.Wact wθ u v2))
368
369
```

Ew, captures interactions between distinct neurons. For each pair u and v2, the function Wact multiplies the connection weight wθ.w u v by the activations of u and v, summing over all pairs to account for the total pairwise influence on the network's energy. The second part

```
370
371 def NeuralNetwork.State.Eθ := ∑ u, θ' (wθ.θ u) * s.act u
372
373
```

represents the contribution of individual neuron thresholds, with each neuron u contributing via wθ.θ u combined with its activation through θ'. The complete energy E is the sum of Ew and Eθ, providing a scalar measure that governs the network's update dynamics.

We show that energy cannot increase during a state transition when a neuron is updated. In an asynchronous update, only one neuron's activation, u, changes from $\text{act}_u^{(\text{old})}$ to $\text{act}_u^{(\text{new})}$. The energy difference between the old and new states consists only of terms containing act_u , with all other terms canceling out. The factor $\frac{1}{2}$ vanishes due to the symmetry of the weights, causing each term to occur twice. Then we can extract the new and old activation states of neuron u from the sums, yielding the energy difference:

Thus, we have:

$$\begin{aligned}
 \Delta E &= E^{(\text{new})} - E^{(\text{old})} \\
 &= \left(- \sum_{v \in U - \{u\}} w_{uv} \text{act}_u^{(\text{new})} \text{act}_v + \theta_u \text{act}_u^{(\text{new})} \right) \\
 &\quad - \left(- \sum_{v \in U - \{u\}} w_{uv} \text{act}_u^{(\text{old})} \text{act}_v + \theta_u \text{act}_u^{(\text{old})} \right) \\
 &= (\text{act}_u^{(\text{old})} - \text{act}_u^{(\text{new})}) \underbrace{\left(\sum_{v \in U - \{u\}} w_{uv} \text{act}_v - \theta_u \right)}_{\text{net}_u}.
 \end{aligned}$$

We consider two cases. If $\text{net}_u < \theta_u$ and the activation changed, then $\text{act}_u^{(\text{new})} = -1$ and $\text{act}_u^{(\text{old})} = 1$, so $\Delta E < 0$. If $\text{net}_u \geq \theta_u$, then $\text{act}_u^{(\text{old})} = -1$ and $\text{act}_u^{(\text{new})} = 1$, yielding $\Delta E \leq 0$.

If a state transition reduces energy, the original state cannot be revisited, as this would require an energy increase. In the second case, where energy remains constant, the transition leads to more +1 activations, as the updated neuron's activation changes from -1 to 1. Thus, the original state cannot be revisited in this case either. The following Lean theorem captures this idea:

```
394
395 theorem energy_lt_zero_or_pluses_incr
396 1 (hc : (s.Up wθ u).act u ≠ s.act u) :
397 2 (s.Up wθ u).E wθ < s.E wθ ∨
398 3 ((s.Up wθ u).E wθ = s.E wθ ∧
399 4 s.pluses < (s.Up wθ u).pluses) := ...
400
401
```

Each state transition reduces the number of reachable states. Since the total number of states is finite, the network will eventually reach a stable state. However, this convergence is only guaranteed if every neuron is eventually updated. If some neurons are excluded, the network may fail to stabilize. To prevent this, we define ‘fairness’ in the update sequence as follows:

```
def fair (useq : ℕ → U) : Prop := ∀ u n, ∃ m ≥ n, useq m = u
```

Then theorem `hopfieldNet_convergence_fair` formalizes the first part of the convergence theorem (Theorem 3.1), corresponding to asynchronous fair updates. It ensures that for any fair update sequence, there exists a time step N after which the network reaches a stable state.

```
theorem hopfieldNet_convergence_fair :
  ∀ (useq : ℕ → U), fair useq →
  ∃ N, (seqStates' s useq N).isStable wθ := ...
```

A fixed cyclic update order naturally ensures that every neuron is updated. We define a cyclic update sequence and prove its inherent fairness using lemma `cycl_Fair` as a sanity check.

```
def cyclic [Fintype U] (useq : ℕ → U) :
  Prop :=
  (∀ u : U, ∃ i, useq i = u) ∧
  (∀ i j, i % card U = j % card U → useq i = useq j)
```

From this cyclic update structure, we derive the following corollary: either traversing all neurons results in no change, indicating a stable state, or at least one activation state changes. In the latter case, one of the 2^n possible states (where n is the number of neurons) is excluded, as the previous state cannot be revisited. Therefore, after at most 2^n neuron traversals, or equivalently $n \cdot 2^n$ updates, a stable state must be reached.

The theorem `hopfieldNet_convergence_cyclic` formalizes the second part of the convergence theorem (Theorem 3.1), corresponding to updates in a fixed cyclic order. Here, `card U` denotes the number of neurons in the network.

```
theorem hopfieldNet_convergence_cyclic :
  ∀ (useq : ℕ → U), cyclic useq →
  ∃ N, N ≤ card U * (2 ^ card U) ∧ (s.seqStates wθ useq N).isStable wθ :=
  ...
```

We’ve shown that asynchronous updates, when applied fairly, always lead the Hopfield network to a stable state, forming the basis for its use in associative memory – a kind of memory that is addressed by its contents.

3.3 Hopfield Networks as Associative Memory

By associative memory we mean a form of memory retrieval in which information is accessed by similarity rather than by an explicit address. In a classical memory model, one typically retrieves stored data by specifying an exact index or key. In contrast, an associative memory can recall a stored item when presented with a partial, noisy, or corrupted version of it. Hopfield networks are well suited for associative memory, leveraging their convergence to stable states. A pattern is a vector of neuron activations representing a stored memory. When presented with an input pattern, the network determines whether it matches a stored pattern. By setting weights and thresholds via the Hebbian algorithm [24] so that desired patterns correspond to stable states, the network can recover the closest stored pattern,

correcting errors and recognizing partially corrupted inputs.

3.4 The Hebbian learning rule

We start by considering how a single pattern $p = (\text{act}_{u_1}, \dots, \text{act}_{u_n})^\top \in \{-1, 1\}^n$, $n \geq 2$, is stored in the network. To this end we have to determine the weights and the thresholds in such a way that the pattern becomes a stable state of the Hopfield network. Therefore we need to ensure $S(Wp - \theta) = p$, where W is the weight matrix of the Hopfield network, $\theta = (\theta_{u_1}, \dots, \theta_{u_n})$ is the threshold vector, and S is a function $S : \mathbb{R}^n \rightarrow \{-1, 1\}^n$, $x \mapsto y$, where the vector y is determined by

$$\forall i \in \{1, \dots, n\} : y_i = \begin{cases} 1 & \text{if } x_i \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

That is, the function S is a kind of element-wise threshold function. If we set $\theta = 0$, that is, if all thresholds are chosen to be zero, a suitable weight matrix W can easily be found. In this case, it clearly suffices if

$Wp = cp$ with $c \in \mathbb{R}_+$. We now choose: $W = pp^\top - I$, where I is the $n \times n$ identity matrix. The term $pp^\top$ is the so-called outer product of p with itself, resulting in a symmetric $n \times n$ matrix. Subtracting the identity matrix ensures that the diagonal entries of W are zero – corresponding to the absence of self-loops in a Hopfield network. With this choice of W , we have: $Wp = (n-1)p$, since $p \in \{-1, 1\}^n$ and $\|p\|^2 = n$. In this form, we first compute the inner product (or scalar product) of the vector p with itself, which gives its squared length. Since $p \in \{-1, 1\}^n$, we have

$$p^\top p = \|p\|^2 = n. \quad (2)$$

Thus, from the earlier computation $Wp = (n-1)p$, we conclude that the eigenvalue $c = n-1$ is strictly positive for $n \geq 2$, as required. Therefore, the pattern p is a stable state of the Hopfield network. This rule is also known as the ‘Hebbian learning rule’ [24].

However, note that this method also makes the pattern $-p$, which is complementary to p , a stable state of the network. The reason is straightforward: if $Wp = (n-1)p$, then it also follows that $W(-p) = (n-1)(-p)$. Unfortunately, this means it is not possible to prevent the network from storing both a pattern and its complement when using this learning rule.

When storing multiple patterns $p_1, \dots, p_m$ with $m < n$, the weight matrix W_i is determined for each pattern p_i as described earlier. The overall weight matrix W is then the sum of these individual matrices:

$$W = \sum_{i=1}^m W_i = \sum_{i=1}^m p_i p_i^\top - mE. \quad (3)$$

In Lean, the Hebbian learning rule for a Hopfield network is implemented as the function `Hebbian`, which, given a collection of m patterns (`ps : Fin m → (HopfieldNetwork R U).State`), returns the corresponding network parameters `Params (HopfieldNetwork R U)`. The threshold vector is set to zero, and the weight matrix is constructed as in Equation (3).

If the patterns to be stored are pairwise orthogonal (that is, if the corresponding vectors are perpendicular to each other), the weight matrix W for an arbitrary pattern p_j , $j \in \{1, \dots, m\}$ is given by:

$$Wp_j = \sum_{i=1}^m p_i (p_i^\top p_j) - mp_j. \quad (4)$$

Since $p_i^T p_j = 0$ for $i \neq j$ and $p_i^T p_i = n$ for $i = j$, the inner product of orthogonal vectors is zero, while the inner product of a vector with itself equals its squared length. Given that $p_j \in \{-1, 1\}^n$, the length of each p_j is n , and hence we conclude that $Wp_j = (n - m)p_j$.

We assume pairwise orthogonal patterns via the following hypothesis, which guarantees that each stored pattern is exactly a stable state:

```
(h : ∀ {i j : Fin m} (h : i ≠ j), dotProduct (ps i).act (ps j).act = 0)
```

The lemma `patterns_pair_orth` encodes Equation (4), showing that for pairwise orthogonal patterns, the weight matrix `(Hebbian ps).w` maps each stored pattern to a scalar multiple of itself. Hence, p_j is a stable state of the Hopfield network if $m < n$. Note that in this case, the complementary pattern $-p_j$ is also a stable state. This is because, as derived earlier, $Wp_j = (n - m)p_j$, and thus we also have $W(-p_j) = (n - m)(-p_j)$. The lemma `hebbian_stable` encodes in Lean that, under the pairwise orthogonality hypothesis and the cardinality condition $m < n$, the pattern p_j is indeed a stable state of the network.

► **Remark 1 (On orthogonality).** In practice, patterns are rarely perfectly orthogonal, and stability then depends on additional disturbance terms.

If the patterns are not pairwise orthogonal we obtain for an arbitrary pattern p_j , where $j \in \{1, \dots, m\}$:

$$Wp_j = (n - m)p_j + \underbrace{\sum_{\substack{i=1 \\ i \neq j}}^m p_i(p_i^T p_j)}_{\text{disturbance term}} . \quad (5)$$

The state p_j may still be stable if $n - m > 0$ and the ‘disturbance term’ is sufficiently small. This happens when the patterns p_i are ‘approximately’ orthogonal, in which case the scalar products $p_i^T p_j$ are small.

Clearly, the more patterns we try to store, the smaller this disturbance term must be. A larger m reduces $n - m$, making the state more susceptible to such disturbances. As a result, in practice, the theoretical limit for the storage capacity of a Hopfield network is rarely achieved.

We will now proceed to perform computations using the Hebbian learning algorithm.

3.5 Hebbian learning computations

We implement the Hebbian learning algorithm for pairwise orthogonal patterns in Lean by converting input patterns into network states, computing the weight matrix, and testing the network’s performance with cyclic and fair update sequences.

We define two patterns `ps` to store in the Hopfield network:

```
def ps : Fin 2 → Fin 4 → ℚ := ![1,1,-1,-1], ![-1,1,-1,1]
```

Next, we use these patterns to define the network parameters, applying the Hebbian learning rule if the patterns are valid:

```
def testParams : Params
  (HopfieldNetwork ℚ (Fin 4)) :=
  match (patternsOfVecs ps (...)) with
  | some patterns => Hebbian patterns
  | none => ZeroParams
```

538 The conversion by the function `patternsOfVecs` does not need to produce patterns
 539 where the activations satisfy the predicate `pact`. For this reason it only produces an `Option`.
 540 However, we not want to have to propagate the `Option` throughout the rest of the example,
 541 and therefore there is a dummy branch in the `match`. The function `testParams` then applies
 542 the Hebbian learning rule to compute the weight matrix.

543 The function `hopfieldNet_stabilize` stabilizes the network to convergence using the
 544 fair update sequence, with `Nat.find` determining the convergence time step. Similarly,
 545 `hopfieldNet_conv_time_steps` calculates the convergence time steps with a cyclic update
 546 sequence, also using `Nat.find`. The function `useqFin` generates a sequence of `Fin n`
 547 neurons, while `useqFin_cyclic` and `useqFin_fair` verify that the sequence is cyclic and
 548 fair, respectively.

```

549
550 1 #eval hopfieldNet_stabilize test_params extu (useq_Fin 4) (useqFin_fair 4)
551 2 --The result shows the final activation states of the network:
552 3 acts: ![-1, 1, -1, 1]
553 4
554 5 #eval hopfieldNet_conv_time_steps test_params extu (useq_Fin 4)
555      (useqFin_cyclic 4)
556 6 --This yields the number of time steps to convergence: 2
557

```

558 Hopfield networks can be described by an energy function: asynchronous updates drive the
 559 system to stable states, with Hebbian learning encoding patterns as such states. While
 560 this deterministic model is mathematically clean, it is also restrictive. To generalize, we
 561 consider stochastic updates, where neurons flip randomly with probabilities determined by
 562 the resulting energy change.

4 Stochastic neural networks

Stochastic updates allow neurons to change state probabilistically. This interpretation links the network’s energy function to probability distributions. The update process can be seen as a Markov Chain Monte Carlo (MCMC) method : a way to sample from a distribution by constructing a Markov chain whose long-run behavior reflects it. The key question is whether repeated updates lead to predictable behavior. In Hopfield networks, asynchronous updates deterministically converge to a stable state. In stochastic networks, ‘stability’ means convergence not to a fixed state but to a stationary distribution. This idea is formalized by the notion of reversibility, or detailed balance. To proceed, we present the necessary groundwork from probability theory.

4.1 Probability theory

Formally, a complete theory of MCMC requires two ingredients: a measure-theoretic definition of the stochastic process and a convergence theory guaranteeing equilibrium. The standard foundation for the first is the Markov kernel, a measurable function specifying transition probabilities over arbitrary state spaces. For the latter, in finite state spaces the kernel reduces to a stochastic matrix, and convergence is characterized by the Perron-Frobenius theorem [40] and the fundamental theorem of Markov chains (Theorem 6.2.2, p. 236 and Theorem 6.2.5, p 237, [40]).

Although `mathlib` provides the foundational infrastructure for Markov kernels, a formal convergence theory for matrices was lacking. We bridge this gap by delivering the first formalization of the Perron-Frobenius theorem in Lean 4, enabling rigorous proofs of ergodicity for Boltzmann machine dynamics – a result previously out of reach. We assume familiarity with basic measure-theoretic probability, including measurable spaces, probability measures, and measurable functions; for further details, see [29], which underlies the constructions in `mathlib`. Most definitions below are presently `noncomputable`, relying on `mathlib`’s nonconstructive real numbers, and thus do not yield executable code.

4.2 Markov kernels

Formally, if $\mathcal{X}$ is the state space a *Markov kernel* (see 4.2.1 (p. 159) [40]) is a function

$$K : \mathcal{X} \times \mathcal{A} \rightarrow [0, 1],$$

where $\mathcal{A}$ is a sigma-algebra of subsets of $\mathcal{X}$. The conditions are:

1. For each fixed state $x \in \mathcal{X}$, the map $A \mapsto K(x, A)$ (with $A \in \mathcal{A}$) is a probability measure on $(\mathcal{X}, \mathcal{A})$.
2. For each $A \in \mathcal{A}$, the map $x \in \mathcal{X} \mapsto K(x, A)$ is measurable.

In the finite-state case, the sigma-algebra $\mathcal{A}$ is simply the full powerset $\mathcal{P}(\mathcal{X})$. Accordingly, a Markov kernel reduces to a *stochastic matrix* (p. 48-52, [42]), where each row represents a probability distribution over the possible states and all entries are nonnegative and sum to one. In this paper, we focus on a finite number of neurons (or variables), so the state space $\mathcal{X}$ is finite and the associated sigma-algebra is exactly $\mathcal{P}(\mathcal{X})$. This simplification is sufficient for our purposes and avoids the complications of infinite-state spaces. We also adopt the convention of *column-stochastic* matrices, where columns sum to one. The entry A_{ij} represents the probability of transitioning from state j to state i .

604 **Single-Site Update Kernel**

605 Let $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_n$ be a finite or countable product state space, representing the states
 606 of n variables. A *single-site update kernel* K_i for site $i \in \{1, \dots, n\}$ is a Markov kernel

$$607 \quad K_i : \mathcal{X} \times \mathcal{P}(\mathcal{X}) \rightarrow [0, 1]$$

608 with the following properties:

- 609 1. For each $x = (x_1, \dots, x_n) \in \mathcal{X}$, $K_i(x, \cdot)$ only modifies the i -th coordinate; that is, for any
 610 measurable set $A \subseteq \mathcal{X}$,

$$611 \quad K_i(x, A) = K_i\left(x, \{y \in \mathcal{X} : y_j = x_j \text{ for } j \neq i, y_i \in A_i\}\right),$$

612 where $A_i = \{y_i : \exists y \in A \text{ with } y_j = x_j \text{ for } j \neq i\}$.

- 613 2. For each fixed $x \in \mathcal{X}$, $K_i(x, \cdot)$ is a probability measure on $\mathcal{X}$.

- 614 3. For each measurable set $A \subseteq \mathcal{X}$, the map $x \mapsto K_i(x, A)$ is measurable.

615 Intuitively, a single-site update kernel ‘resamples’ only one coordinate of the system while
 616 keeping the others fixed, which underlies neuron updates in stochastic finite-state systems
 617 such as Boltzmann machines.

618 **Gibbs Updates**

619 To generate samples from a distribution π , we define single-site Gibbs updates (see 7.1.1,
 620 p. 285, [40]). Let $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_n$ be a finite product state space, and let π be a target
 621 probability measure on $\mathcal{X}$. For each site $i \in \{1, \dots, n\}$, the *Gibbs update kernel* K_i is a
 622 single-site kernel defined by

$$623 \quad K_i(x, A) = \sum_{x'_i \in \mathcal{X}_i : (x_1, \dots, x_{i-1}, x'_i, x_{i+1}, \dots, x_n) \in A} \pi(x'_i \mid x \setminus i),$$

624 for all measurable sets $A \subseteq \mathcal{X}$, where $x \setminus i$ denotes all coordinates of x except the i -th. We
 625 implement this kernel (Algorithm A.31, p. 301 [40]) using Lean’s discrete probability mass
 626 function (PMF) monad. The implemented algorithm, `gibbsUpdate`, is a direct formalization of
 627 Gibbs sampling. Mathematically, Gibbs sampling is a special case of the Metropolis-Hastings
 628 (MH) algorithm [23]. In MH, one proposes a move and accepts it with a certain probability
 629 that ensures invariance of the target distribution. In Gibbs sampling, the proposal is given
 630 by the exact conditional distribution of a single coordinate given all the others. For this
 631 particular choice, the MH acceptance probability is always 1.

632 The structure of the random process is defined constructively using the `Probability`
 633 `MassFunction` (PMF) type, consistently with the main choice of structuring our neural
 634 network in a linearly ordered field. However, the numerical probabilities passed to the
 635 constructor in `gibbsUpdate` depend on `Real.exp` and is therefore **noncomputable**.

636 Although working with kernels does not yet give us the full convergence theory of Markov
 637 chains, this approach is enough to describe the stochastic dynamics of neural networks. The
 638 central question is: under repeated updates, does the network reach a stable, predictable
 639 behavior? In stochastic networks, ‘stabilization’ no longer means convergence to a single state
 640 but convergence to a stationary distribution over states. The mathematical concept that
 641 captures this notion is *reversibility*, also known as detailed balance in the physics literature.

4.3 Reversibility and Ergodicity

We follow (p. 235, [40]) for definitions presented in this section. A Markov kernel K_i is *reversible* (def. 6.2.1, [40]) with respect to a probability measure π on $\mathcal{X}$ if it satisfies the detailed balance condition

$$\pi(dx) K_i(x, dy) = \pi(dy) K_i(y, dx),$$

or equivalently, for all measurable sets $A, B \subseteq \mathcal{X}$,

$$\int_A K_i(x, B) d\pi(x) = \int_B K_i(x, A) d\pi(x).$$

Reversibility means that at equilibrium, the ‘probability flow’ between any two states s and s' is symmetric: the probability of being in s times the transition probability to s' equals the reverse flow. If this holds for all such pairs, the state distribution is stable.

If each single-site kernel K_i is reversible with respect to π , the *random-scan kernel* (`randomScanKernel`),

$$K_{\text{rand}}(x, A) = \frac{1}{n} \sum_{i=1}^n K_i(x, A)$$

is also reversible with respect to π .

In general, given a Markov kernel K on a state space, a probability measure π is called *invariant* if applying the kernel does not change the measure. That is, if the system is initially distributed according to π , then after one step of the kernel the distribution remains π , and the same holds after any number of steps.

Reversibility implies that π is invariant under the kernel (Theorem 6.2.2, [40]): for all $A \subseteq \mathcal{X}$,

$$\pi(A) = \int_{\mathcal{X}} K_i(x, A) d\pi(x),$$

which ensures that π is a stationary distribution for both single-site updates and their uniform mixture. The *random-scan kernel* K_{rand} thus provides a canonical way to construct a reversible Markov kernel for the full system from its single-site components.

4.4 Perron-Frobenius theorem

While reversibility guarantees that the distribution π is invariant, it does not guarantee that the system will actually converge to π from an arbitrary starting state, nor that π is unique. This stronger property is known as *ergodicity*. For a finite state space, a Markov chain is ergodic (Theorem 6.2, [40]) if it is both *irreducible* (every state is reachable from every other state) and *aperiodic* (it does not get trapped in deterministic cycles).

The convergence of finite Markov chains is formalized by the Perron-Frobenius theorem for non-negative matrices. A non-negative matrix A is defined as *irreducible* if for every pair of indices (i, j) , there exists a power k such that the (i, j) -th entry of A^k is positive ([42], Definition 1.6, p. 29).

► **Theorem 2** (Theorem 1.5). (*pft_irreducible*) *LinearMap.onSup* $\mathbb{R}$ Suppose T is an $n \times n$ irreducible non-negative matrix. Then there exists an eigenvalue r such that:

- (a) r is real and $r > 0$;
 - (b) (*ev_of_primitive_is_pos*) with r can be associated strictly positive left and right eigenvectors;
 - (c) (*perron_root_is_spec_radius*) $r \geq |\lambda|$ for any eigenvalue λ of T ;
 - (d) (*geometric_mult_one*) the eigenvectors associated with r are unique up to constant multiples;
 - (e) if $0 \leq B \leq T$ and β is an eigenvalue of B , then $|\beta| \leq r$. Moreover, $|\beta| = r$ implies $B = T$;
 - (f) (*algebraic_mult_one*) r is a simple root of the characteristic equation of T .
- Corollaries 1 and 2 of Theorem 1.1 hold also.

The theorem states that an irreducible non-negative matrix has a positive real eigenvalue, known as the Perron root, which is equal to its spectral radius ([42], Theorem 1.5, p. 33). Associated with this eigenvalue is a unique eigenvector (up to scaling) with strictly positive components. For a stochastic matrix, which is non-negative and has row sums equal to one, the Perron root is always 1. The corresponding left eigenvector is the unique stationary distribution.

The theorem further distinguishes between aperiodic and periodic matrices. If an irreducible matrix is aperiodic (or *primitive*), the Perron root is the unique eigenvalue of maximum modulus ([42], Theorem 1.1, p. 14). If it is periodic with period $d > 1$, there are exactly d eigenvalues with modulus equal to the Perron root ([42], Theorem 1.7, p. 34). This distinction is essential for ergodicity as aperiodicity ensures convergence to a unique stationary distribution, whereas periodicity results in cyclic behavior.

Our formal proof of ergodicity for Boltzmann machines relies on a new formalization of this theorem.

Design decisions for PF

The key design choice was to ground the concept of irreducibility in graph theory by leveraging and substantially contributing to `mathlib`'s Quiver library.

We define a function `Matrix.toQuiver` that maps a matrix A to a directed graph, where an edge $i \rightarrow j$ exists if and only if $A_{ij} > 0$. Irreducibility is then defined as the strong connectivity of this graph.

```

1 def Irreducible (A : Matrix n n ℝ) : Prop :=
2   (∀ i j, 0 ≤ A i j) ∧ IsStronglyConnected (toQuiver A)

```

This combinatorial foundation is linked to the algebraic properties of the matrix via the lemma `pow_to_path`, which proves that a positive entry in A^k is equivalent to the existence of a path of length k . This allows us to prove essential properties, such as the strict positivity of eigenvectors of irreducible matrices (`eig_prim_pos`).

The analytical part of the proof (see p. 15 and Theorem 1.1 [42]) centers on the Collatz-Wielandt function:

► **Definition 3.** (*collatzWielandtFn*) Let T be a non-negative $n \times n$ matrix. The Perron root r of T can be characterized by the Collatz-Wielandt formula which defines the Perron root `perronRoot_alt` as $r = \sup_{\mathbf{x} \geq 0, \mathbf{x} \neq \mathbf{0}} \left(\min_i \text{ s.t. } x_i > 0 \frac{(T\mathbf{x})_i}{x_i} \right)$.

The primary formalization challenge was that this function is not continuous where vector components are zero.

We addressed this by proving it is upper-semicontinuous on the compact standard simplex `compact_simplex`, enabling the use of a generalized Extreme Value Theorem `exists_max_usco` to guarantee the existence of a maximizer (`exists_max`), which is then proven to be the Perron eigenvector via the (`max_is_eig`). With this lemma we can prove the Perron-Frobenius theorem (Theorem 1.5, p. 33-34, [42]) for stochastic matrices:

► **Theorem 4.** (*colStIrrdSmlpxEgn_unique*) Let A be an $n \times n$ column-stochastic, irreducible matrix. Then there exists a unique probability vector v (i.e., $v_i \geq 0$ for all i and $\sum_i v_i = 1$) such that $A\mathbf{v} = \mathbf{v}$.

This vector v is the unique stationary distribution of the Markov chain defined by the transition matrix A . For the Boltzmann machine, we prove its stochastic matrix `RScol` is irreducible (`RScol_irred`) and aperiodic (`RScol_diagPos`), and then apply this theorem to guarantee the existence of a unique stationary distribution (`RScol_uniqStatSmlpx`), completing the proof of ergodicity.

Design for PF

A unique feature of this formalization is that the development of the underlying mathematical library actively reshaped the definition of the application layer. Initially, we modeled neural networks as simple directed graphs, aligning with standard physics intuition where the "network topology" and "weight matrix" are treated as interchangeable concepts. However, the formalization of the Perron-Frobenius theorem (Section 4) exposed a critical semantic gap in this informal view.

To prove ergodicity for Boltzmann machines, we required a rigorous characterization of matrix irreducibility. We found that while irreducibility is defined algebraically via non-negative powers ($A^k > 0$), it is reasoned about combinatorially via strong connectivity. Attempting to bridge these views with simple graphs created significant friction. This led us to the Quiver (directed multidigraph) interface in `mathlib`, which provides a generalized theory of paths and preorders. As we integrated this infrastructure, we realized that Quiver was not merely a tool for the proofs, but the correct fundamental abstraction for the networks themselves.

Consequently, we redesigned the `NeuralNetwork` structure to natively extend Quiver (via `Digraph`). This design choice unifies the two faces of the network: we treat weights as algebraic data attached to the quiver's edges-preserving the Matrix correspondence for computation-while inheriting the rich path logic of category theory for topological proofs. This evolution highlights a distinct advantage of formalization: the rigorous requirements of

756 the general theory (Perron-Frobenius) forced us to discover a more robust and generalized
757 architectural definition than the one typically found in physics literature.

With single-site kernels and stochastic updates established, we now turn to Boltzmann machines, whose energy function ensures that Gibbs updates satisfy detailed balance with respect to the Boltzmann distribution.

4.5 Boltzmann machines

Boltzmann machines are a canonical example of stochastic networks (Section 8.6, [30]). The energy function is identical to that of Hopfield networks (Equation (1)) :

$$E(\mathbf{act}) = -\frac{1}{2}\mathbf{act}^T W \mathbf{act} + \boldsymbol{\theta}^T \mathbf{act}, \quad (6)$$

where $\mathbf{act} = (\text{act}_{u_1}, \dots, \text{act}_{u_n})^\top \in \{-1, 1\}^n$, $n \geq 2$, is the state of the neuron activations, W the weight matrix, and $\boldsymbol{\theta} = (\theta_{u_1}, \dots, \theta_{u_n})$ the threshold vector.

In our setting, the target distribution is the *Boltzmann distribution*

$$\pi(\mathbf{act}) = \frac{1}{c} \exp\left(-\frac{E(\mathbf{act})}{kT}\right) \quad (7)$$

where $c = \sum_{\mathbf{act}} \exp(-E(\mathbf{act})/T)$ ensures the probabilities of all possible states equal one, a real number $T > 0$ denoting the temperature of the system, and k is Boltzmann's constant[†]. We adopt the common simplification in machine learning by replacing kT with T , omitting Boltzmann's constant.

Following the approach in §2, we now focus on a single neuron u and examine how changing its state affects the network's energy. For clarity, we adopt a notation that allows the reader to interpret updates in terms of probabilities rather than kernels. For a network state $\mathbf{act}$ and a neuron u , we write $\mathbb{P}(\text{act}_u = \text{act}_u^{(\text{new})})$ as shorthand for the conditional probability $\mathbb{P}(\text{act}_u = \text{act}_u^{(\text{new})} \mid \mathbf{act}_{\setminus u})$, where $\mathbf{act}_{\setminus u}$ denotes the activations of all neurons except u .

This conditional probability fully specifies the single-site Gibbs update: neuron u is set to $\text{act}_u^{(\text{new})}$ with probability $\mathbb{P}(\text{act}_u = \text{act}_u^{(\text{new})})$, while all other activations remain unchanged. In other words, one can read the update rules directly as probabilities for individual neurons, without explicitly referring to the kernel notation K_u .

We now consider the absolute difference in energy between the two possible states of u , $\text{act}_u = -1$ and $\text{act}_u = +1$, while keeping the states of all other units fixed. This difference is

$$\Delta E_u = E_{\text{act}_u=1} - E_{\text{act}_u=-1} = 2 \sum_{v \in U \setminus \{u\}} w_{uv} \text{act}_v - \theta_u. \quad (8)$$

Writing the energies in terms of the Boltzmann distribution (Equation (7)) yields

$$\Delta E_u = -T \ln(\mathbb{P}(\text{act}_u = +1)) + T \ln(\mathbb{P}(\text{act}_u = -1)), \quad (9)$$

since the probability of u having activation -1 and the probability of it having activation 1 must sum to 1 (since there are only these two possible states). Solving for the conditional probability $\mathbb{P}(\text{act}_u = 1)$ of neuron u being active gives

$$\mathbb{P}(\text{act}_u = +1) = \frac{1}{1 + e^{-\Delta E_u/2T}}, \quad (10)$$

[†] $k \approx 1.38 \cdot 10^{-23} \text{ J/K}$, that is, the unit is Joule (energy) divided by Kelvin (temperature).

a logistic function of the (scaled) energy difference between its active and inactive states. Because the energy difference is directly linked to the network input,

$$\Delta E_u = 2 \sum_{v \in U \setminus \{u\}} w_{uv} \text{act}_v - \theta_u = 2(\text{net}_u - \theta_u), \quad (11)$$

this formula suggests a stochastic update rule for the neuron. The stochastic update procedure is as follows: a neuron u is selected at random. Its energy difference ΔE_u is computed based on the current state of all other neurons, $\mathbf{act}_{\setminus u}$, and the conditional probability of activation $\mathbb{P}(\text{act}_u = +1)$ is evaluated. The neuron is then set to $+1$ with this probability and to -1 otherwise. Concretely, this can be implemented by sampling a random number x uniformly from the interval $[0, 1]$ and updating the neuron according to

$$\text{act}_u^{(\text{new})} = \begin{cases} +1, & \text{if } x \leq \mathbb{P}(\text{act}_u = +1), \\ -1, & \text{otherwise.} \end{cases} \quad (12)$$

This update is repeated many times for randomly chosen neurons. As a consequence, Boltzmann machines can be seen representations of and sampling mechanisms for the Boltzmann distributions defined by their connection weights and threshold values.

As shown in §4.2, single-site Gibbs updates satisfy detailed balance with respect to the network energy, a property exemplified by Boltzmann machines.

Specifically, the conditional probability that neuron u takes a new state $\text{act}_u^{(\text{new})}$ given the current states of all other neurons $\mathbf{act}_{\setminus u}$ is

$$\mathbb{P}(\text{act}_u = \text{act}_u^{(\text{new})}) = \frac{\exp(-E(\mathbf{act}_{\setminus u}, \text{act}_u^{(\text{new})})/T)}{\sum_{s \in \{+1, -1\}} \exp(-E(\mathbf{act}_{\setminus u}, s)/T)}. \quad (13)$$

This conditional probability fully characterizes the single-site Gibbs update, which by construction satisfies detailed balance.

$$\pi(\mathbf{act}) \mathbb{P}(\text{act}_u^{(\text{new})} = \text{act}_u^{(\text{new})}) = \pi(\mathbf{act}_{[u \rightarrow \text{act}_u^{(\text{new})}]}) \mathbb{P}(\text{act}_u^{(\text{new})} = \text{act}_u), \quad (14)$$

where $\mathbf{act}_{[u \rightarrow \text{act}_u^{(\text{new})}]}$ denotes the network state obtained by replacing the activation of neuron u in $\mathbf{act}$ with $\text{act}_u^{(\text{new})}$, leaving all other neurons unchanged.

Since we already established via detailed balance that the Boltzmann distribution is a stationary distribution, the uniqueness guaranteed by the Perron-Frobenius theorem ensures it is the only one, and the system is ergodic. The theorem `randomScan_ergodicUniqueInvariant` summarizes these properties: reversibility, aperiodicity, irreducibility, and uniqueness [21].

This construction generalizes Hopfield network dynamics. At high temperature, updates are stochastic, allowing neurons to flip even if energy increases; as $T \rightarrow 0$, updates become deterministic, recovering the asynchronous Hopfield dynamics. As shown in `gibbs_zero_temp_limit`, the one-site Gibbs update probability mass function converges pointwise to the zero-temperature limit kernel as $1/T \rightarrow \infty$.

The key insight, that a single neuron's update rule can be derived directly from the network's energy function, highlights that the Boltzmann distribution is not merely a probabilistic construct, but the fundamental object describing systems in equilibrium in statistical mechanics (see Example 5.2.5, p. 201, [40]).

Tasks and Learning Objectives

Unlike Hopfield networks, which rely on deterministic convergence to local energy minima, the computational role of a Boltzmann machine is defined by the properties of its stationary

distribution $\pi(\mathbf{act})$ defined in (7). This probabilistic formulation supports two distinct tasks depending on how the system is utilized.

- **Stochastic Associative Memory:** As previously discussed, the temperature T controls the randomness of the system. In the low-temperature regime, the probability mass of $\pi(\mathbf{act})$ concentrates around the global minima of the energy function $E(\mathbf{act})$. In this setting, the system functions as an associative memory similar to a Hopfield network. Retrieval corresponds to clamping a subset of neurons (fixing their values within $\mathbf{act}$) and allowing the remaining neurons to relax towards equilibrium via single-site Gibbs updates. This stochasticity provides a robust mechanism for recall, allowing the system to escape shallow local minima that would trap a deterministic network.
- **Unsupervised Density Estimation:** In the general setting, the objective is not merely to find energy minima, but to shape the energy landscape such that the stationary distribution $\pi(\mathbf{act})$ approximates an empirical distribution observed in a dataset[cite: 3]. Learning consists of tuning the weights W and thresholds θ to maximize the log-likelihood of the data. Differentiating the log-likelihood yields the characteristic gradient update rule:

$$\Delta w_{uv} \propto \underbrace{\mathbb{E}_{\text{data}}[\text{act}_u \text{act}_v]}_{\text{Positive Phase}} - \underbrace{\mathbb{E}_{\pi}[\text{act}_u \text{act}_v]}_{\text{Negative Phase}}. \quad (15)$$

Here, the ‘Positive Phase’ represents the correlation between neurons u and v when the network is clamped to training data, while the ‘Negative Phase’ represents the correlation generated by the model’s own stationary distribution $\pi(\mathbf{act})$. The learning process effectively minimizes the discrepancy between the data statistics and the model statistics.

Both tasks fundamentally rely on the convergence of the stochastic dynamics to the unique stationary distribution $\pi(\mathbf{act})$. Our current formalization establishes this rigorous foundation by proving that the single-site Gibbs update kernel is ergodic and possesses $\pi(\mathbf{act})$ as its unique invariant measure (Section 4.6). The formal derivation of the learning rule in (15) and the verification of algorithmic approximations, such as Contrastive Divergence, remain distinct challenges reserved for future work.

The task of a Boltzmann machine is probabilistic: to represent and manipulate a target distribution over network states. In particular, given data samples, the goal is to learn parameters such that the stationary distribution of the network approximates the empirical data distribution [?].

4.6 Integration with PhysLean

PhysLean [47] formalizes key physics definitions, computations, and theorems in Lean 4, aiming for a comprehensive library akin to **mathlib**, covering classical mechanics, quantum mechanics, electrodynamics, statistical mechanics, and more. Our work integrates with **PhysLean** by interpreting the network energy as a Hamiltonian H a function assigning an energy value to each network state. For two-state networks §2.1, the Hamiltonian matches the network’s energy. Stochastic updates then correspond to sampling from a *canonical ensemble*, a standard concept in statistical mechanics already formalized in **PhysLean**. Canonical ensembles (p. 251, [17]) describe a system in thermal equilibrium at temperature $T > 0$:

$$\pi(x) = \frac{e^{-H(x)/T}}{\sum_{y \in \mathcal{X}} e^{-H(y)/T}}, \quad (16)$$

favoring lower-energy states.

The `IsHamiltonian` typeclass links `NeuralNetwork` to `CanonicalEnsemble`, embedding neural networks into statistical mechanics and enabling the reuse of `PhysLean`’s codebase to formalize noise tolerance, convergence, and other deeper properties of learning in both deterministic and stochastic settings.

5 Design decisions

A fundamental design decision in formalizing neural networks is the choice of the underlying representation. Modern deep learning frameworks, such as TensorFlow [2] and PyTorch [39], predominantly adopt a sequential model, in which networks are represented as a sequence of layers – typically a list paired with an abstract table of activation functions – or, more generally, as a directed acyclic graph (DAG) of tensor operations, i.e., a computational graph. This paradigm has strongly influenced existing formalization efforts.

Formalizations following this perspective, such as the sequential models in Isabelle/HOL [15] and the `kernel` representation in Rocq’s MLCERT [9], emphasize the data flow and transformations applied at each layer. This approach is particularly well-suited to verification goals that closely reflect real-world implementations, where interoperability with external frameworks is a practical consideration. Prior work in Isabelle/HOL [15] explored dual encodings – one graph-based for proofs and one for computation – enabling formal reasoning while supporting on-demand execution in frameworks such as TensorFlow. Inspired by this approach, Rocq [6] incorporated external tools, including MILP and SMT solvers, to support similar verification objectives. Regarding interoperability, [9] relied on an unverified Python script to import models, while [15] required proving correctness for each TensorFlow import. The sequential model is also well-suited for verifying localized computations, such as tensor operations or the correctness of backpropagation, as demonstrated in Certigrad [41]. In proof assistants that enforce provable termination, such as Rocq, feedforward networks can be represented inductively and evaluated via structural recursion, which automatically ensures termination [9]. However, the sequential layer paradigm is limited in capturing arbitrary network topologies, especially the cycles present in recurrent networks. Even more general computational graphs (DAGs) often require complex unfolding or fixed-point semantics to faithfully model true recurrence.

Because our focus is on the global dynamic properties of recurrent models – specifically convergence and ergodicity – rather than individual computational paths, we adopt a graph-based approach. This formalization, fully integrated with `mathlib` and `PhysLean`, is built on `mathlib`’s `Digraph` structure (§2), which suffices for fully connected topologies and enables natural reasoning about global properties such as energy minimization (§3) and Markov chain irreducibility (§4). The main trade-off is reduced computational efficiency, as noted in Section §2.

6 Related Work

Theorem proving has been applied to machine learning, and there is extensive work on using automated theorem provers to verify neural networks [14, 16, 20, 48, 31, 32]. In contrast, the formalization of learning algorithms themselves remains relatively unexplored. In addition to the related work already mentioned in §5, the work most closely related to ours in convergence is [36], where Murphy et al. formalized and proved convergence of the one-layer perceptron and its averaged variant in Rocq. They evaluated performance by extracting

code to Haskell and comparing it with both an arbitrary-precision C++ implementation and a hybrid approach that certified learned separators in Rocq. In Lean 4, by contrast, separate extraction is unnecessary: programs can be written, verified, and compiled in the same environment, with proofs in `Prop` erased at runtime, incurring no overhead.

Beyond convergence results, only a few other machine learning formalisations exist. PAC learnability for decision stumps has been formalized in Lean [49, 44]; Bentkamp et al. [10] formalized the expressiveness of deep learning in Isabelle/HOL; Abdulaziz et al. [3] verified AI planning via a SAT encoder; Vajjha et al. [51, 50] formalized Dvoretzky’s convergence theorem and value/policy iteration in Rocq; and expectation and support vector machines have also been mechanized in Rocq [11].

Probability theory

Probability theory has been formalized in several proof assistants: Doob’s martingale convergence theorems have been mechanized in Lean [52]. HOL4 formalizes limit theorems and verified models such as hidden Markov chains [37, 33]; Mizar covers discrete probability [38]; and Rocq includes Shannon’s theorems and logics for randomized concurrency [5, 43]. Isabelle/HOL formalizes measure-theoretic results, including the central limit theorem and the law of large numbers [8, 19, 26], and supports Markov kernels via the Girny monad and the Ionescu-Tulcea extension theorem for constructing process measures [25].

Perron-Frobenius theorem

The most extensive prior formalization of the Perron–Frobenius theorem was by Thiemann et al. [45] in Isabelle/HOL. While both projects formally prove this central result in matrix theory, our work represents a distinct formalization, differing in mathematical approach, library architecture, and overall contributions.

The two formalizations rely on different mathematical arguments. The Isabelle/HOL proof is topological, using Brouwer’s fixed-point theorem to establish the Perron eigenvector. In contrast, our work provides, to our knowledge, the first formalization of the analytical proof via the Collatz–Wielandt formula (3, including supporting results of independent interest, such as the upper-semicontinuity of the Collatz–Wielandt function and the identification of its maximizer on the standard simplex with the Perron eigenvector. Our subsequent proofs of uniqueness and spectral dominance are also novel, following an analytical approach based on the phase alignment of complex eigenvectors, distinct from the topological method.

The surrounding proof assistant ecosystem shaped each project’s strategy. Isabelle/HOL notably bridged two distinct matrix libraries ([22, 46]) through sophisticated engineering. In contrast, our formalization is entirely within `mathlib`, where analysis, algebra, and combinatorics are integrated. This cohesion allows a different approach, including a novel formalization of matrix irreducibility via the strong connectivity of the associated quiver, formally proven equivalent to the standard algebraic characterization.

The two projects cover different aspects of the Perron-Frobenius theorem. Isabelle/HOL has a broader scope, analyzing the full peripheral spectrum and reducible matrices for applications in program analysis. In contrast, our work, motivated by Markov chain ergodicity, focuses on irreducible matrices, proving existence, uniqueness, and dominance of the Perron root and its positive eigenvector.

7 Conclusion and future work

This paper presents a graph-based framework for neural networks that accommodates Hopfield networks and Boltzmann machines. Our development shows that formal proof assistants can faithfully capture the interplay between the combinatorial, probabilistic and thermodynamic aspects of neural networks, and thereby establishing a verified bridge between statistical mechanics, machine learning, and formal mathematics. Looking ahead, several natural directions for future work emerge.

On the probabilistic side, one can extend the formalization of Markov chain theory and MCMC by developing a general MCMC library. This requires completing the Perron-Frobenius theorem for general non-negative matrices and the Fundamental Theorem of Markov Chains in `mathlib`, providing a foundation for advanced convergence analysis [35], including verification of sophisticated sampling algorithms such as Metropolis-Hastings [23] and simulated annealing (p.142, Example 4.2.4, [40]) beyond the single-site Gibbs updates formalized here.

On the computational side, while our current formalization of discrete Hopfield networks can compute, its efficiency is not practical. However, it is suitable as a target specification for verification of a realistic implementation using refinement. This implementation might use Lean 4, but could also be based on a more hardware-oriented framework outside of Lean. Similarly, making the development fully computable is also a promising direction. In Lean, this would require using a computable reals library analogous to those developed in Rocq such as Coquelicot’s real analysis [12], Flocq’s floating-point library [13], or the constructive reals in CoRN [18].

References

- 1 The Nobel Prize in Physics 2024. <https://www.nobelprize.org/prizes/physics/2024/press-release/>. Accessed: 2025-1-5.
- 2 Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A System for Large-Scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI’16, page 265–283, USA, 2016. USENIX Association.
- 3 Mohammad Abdulaziz and Friedrich Kurz. Verified SAT-Based AI Planning. *Archive of Formal Proofs*, October 2020. https://isa-afp.org/entries/Verified_SAT_Based_AI_Planning.html, Formal proof development.
- 4 David H. Ackley, Geoffrey E. Hinton, and Terrence J. Sejnowski. A Learning Algorithm for Boltzmann Machines. *Cognitive Science*, 9(1):147–169, 1985. doi:10.1016/S0364-0213(85)80012-4.
- 5 Reynald Affeldt and Manabu Hagiwara. Formalization of Shannon’s Theorems in SSReflect-Coq. In *3rd Conference on Interactive Theorem Proving (ITP 2012)*, volume 7406 of *Lecture Notes in Computer Science*, pages 233–249. Springer, 2012.
- 6 Andrei Aleksandrov and Kim Völlinger. Formalizing Piecewise Affine Activation Functions of Neural Networks in Coq. In *NASA Formal Methods: 15th International Symposium, NFM 2023, Houston, TX, USA, May 16–18, 2023, Proceedings*, page 62–78, Berlin, Heidelberg, 2023. Springer-Verlag. doi:10.1007/978-3-031-33170-1_4.
- 7 Britt Anderson. Computational neuroscience in Lean?, September 2024. Zulip message. URL: <https://leanprover.zulipchat.com/#narrow/channel/113488-general/topic/Computational.20Neuroscience.20in.20Lean.3F>.

- 1005 **8** Jeremy Avigad, Johannes Hölzl, and Luke Serafin. A Formally Verified Proof of the Central
1006 Limit Theorem. *Journal of Automated Reasoning*, 59(4):389–423, 2017.
- 1007 **9** Alexander Bagnall and Gordon Stewart. Certifying the True Error: Machine Learning in Coq
1008 with Verified Generalization Guarantees. *Proceedings of the AAAI Conference on Artificial
1009 Intelligence*, 33(01):2662–2669, Jul. 2019. URL: [https://ojs.aaai.org/index.php/AAAI/
1010 article/view/4115](https://ojs.aaai.org/index.php/AAAI/article/view/4115), doi:10.1609/aaai.v33i01.33012662.
- 1011 **10** Alexander Bentkamp, Jasmin Christian Blanchette, and Dietrich Klakow. A Formal Proof of
1012 the Expressiveness of Deep Learning. *Journal of Automated Reasoning*, 63:347–368, 2019.
- 1013 **11** Sooraj Bhat. *Syntactic Foundations for Machine Learning*. PhD thesis, Georgia Institute of
1014 Technology, 2013.
- 1015 **12** Sylvie Boldo, Catherine Lelay, and Guillaume Melquiond. Coquelicot: A User-Friendly Library
1016 of Real Analysis for Coq. *Mathematics in Computer Science*, 9(1):41–62, March 2015. URL:
1017 <https://inria.hal.science/hal-00860648>, doi:10.1007/s11786-014-0181-1.
- 1018 **13** Sylvie Boldo and Guillaume Melquiond. Flocq: A Unified Library for Proving Floating-Point
1019 Algorithms in Coq. In *2011 IEEE 20th Symposium on Computer Arithmetic*, pages 243–252,
1020 2011. doi:10.1109/ARITH.2011.40.
- 1021 **14** Elena Botoeva, Panagiotis Kouvaros, Jan Kronqvist, Alessio Lomuscio, and Ruth Misener.
1022 Efficient Verification of ReLU-Based Neural Networks via Dependency Analysis. *Proceedings
1023 of the AAAI Conference on Artificial Intelligence*, 34(04):3291–3299, Apr. 2020. URL: [https:
1024 //ojs.aaai.org/index.php/AAAI/article/view/5729](https://ojs.aaai.org/index.php/AAAI/article/view/5729), doi:10.1609/aaai.v34i04.5729.
- 1025 **15** Achim D. Brucker and Amy Stell. Verifying Feedforward Neural Networks for Classification
1026 in Isabelle/HOL. In *Formal Methods: 25th International Symposium, FM 2023, Lübeck,
1027 Germany, March 6–10, 2023, Proceedings*, page 427–444, Berlin, Heidelberg, 2023. Springer-
1028 Verlag. doi:10.1007/978-3-031-27481-7_24.
- 1029 **16** Rudy R Bunel, Ilker Turkaslan, Philip Torr, Pushmeet Kohli, and Pawan K Mudi-
1030 gonda. A Unified View of Piecewise Linear Neural Network Verification. In S. Ben-
1031 gio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, ed-
1032 itors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associ-
1033 ates, Inc., 2018. URL: [https://proceedings.neurips.cc/paper_files/paper/2018/file/
1034 be53d253d6bc3258a8160556dda3e9b2-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2018/file/be53d253d6bc3258a8160556dda3e9b2-Paper.pdf).
- 1035 **17** James P. Coughlin and Robert H. Baran. *Neural Computation in Hopfield Networks and
1036 Boltzmann Machines*. Rowman & Littlefield, 1995. URL: [https://api.semanticscholar.
1037 org/CorpusID:59952015](https://api.semanticscholar.org/CorpusID:59952015).
- 1038 **18** Luís Cruz-Filipe, Herman Geuvers, and Freek Wiedijk. C-CoRN, the Constructive Coq
1039 Repository at Nijmegen. In Andrea Asperti, Grzegorz Bancerek, and Andrzej Trybulec, editors,
1040 *Mathematical Knowledge Management*, pages 88–103, Berlin, Heidelberg, 2004. Springer Berlin
1041 Heidelberg.
- 1042 **19** Manuel Eberl. The Laws of Large Numbers. *Archive of Formal Proofs*, 2021. [https://isa-
1043 afp.org/entries/Laws_of_Large_Numbers.html](https://isa-afp.org/entries/Laws_of_Large_Numbers.html).
- 1044 **20** Rüdiger Ehlers. Formal Verification of Piece-Wise Linear Feed-Forward Neural Networks. In
1045 Deepak D’Souza and K. Narayan Kumar, editors, *Automated Technology for Verification and
1046 Analysis*, pages 269–286, Cham, 2017. Springer International Publishing.
- 1047 **21** Stuart Geman and Donald Geman. Stochastic Relaxation, Gibbs Distributions, and the
1048 Bayesian Restoration of Images. *IEEE Transactions on Pattern Analysis and Machine
1049 Intelligence*, PAMI-6(6):721–741, 1984. doi:10.1109/TPAMI.1984.4767596.
- 1050 **22** John Harrison. The HOL Light Theory of Euclidean Space. *Journal of Automated Reasoning*,
1051 50, 02 2013. doi:10.1007/s10817-012-9250-9.
- 1052 **23** W. K. Hastings. Monte Carlo Sampling Methods Using Markov Chains and Their Applications.
1053 *Biometrika*, 57(1):97–109, 04 1970. arXiv:[https://academic.oup.com/biomet/article-pdf/
1054 57/1/97/23940249/57-1-97.pdf](https://academic.oup.com/biomet/article-pdf/57/1/97/23940249/57-1-97.pdf), doi:10.1093/biomet/57.1.97.
- 1055 **24** Donald O. Hebb. *The organization of behavior: A neuropsychological theory*. Wiley, New York,
1056 June 1949.

- 1057 25 Johannes Hölzl. Markov Processes in Isabelle/HOL. In *Proceedings of the 6th ACM SIGPLAN*
 1058 *Conference on Certified Programs and Proofs, CPP 2017*, page 100–111, New York, NY, USA,
 1059 2017. Association for Computing Machinery. doi:10.1145/3018610.3018628.
- 1060 26 Johannes Hölzl, Andreas Lochbihler, and Dmitriy Traytel. A Zoo of Probabilistic Systems.
 1061 *Archive of Formal Proofs*, 2015. [https://isa-afp.org/entries/Probabilistic_System_Zoo.](https://isa-afp.org/entries/Probabilistic_System_Zoo.html)
 1062 [html](https://isa-afp.org/entries/Probabilistic_System_Zoo.html).
- 1063 27 John J Hopfield. Neural Networks and Physical Systems with Emergent Collective Computa-
 1064 tional Abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.
- 1065 28 Ernst Ising. Contribution to the Theory of Ferromagnetism. *Z. Phys.*, 31:253–258, 1925.
 1066 doi:10.1007/BF02980577.
- 1067 29 Olav Kallenberg. *Foundations of Modern Probability*. Probability and its Applications (New
 1068 York). Springer-Verlag, New York, second edition, 2002. URL: [http://dx.doi.org/10.1007/](http://dx.doi.org/10.1007/978-1-4757-4015-8)
 1069 [978-1-4757-4015-8](http://dx.doi.org/10.1007/978-1-4757-4015-8), doi:10.1007/978-1-4757-4015-8.
- 1070 30 Rudolf Kruse, Sanaz Mostaghim, Christian Borgelt, Christian Braune, and Matthias Stein-
 1071 brecher. *Computational Intelligence - A Methodological Introduction, 3rd edition*. Springer
 1072 Verlag, March 2022. doi:10.1007/978-3-030-42227-1.
- 1073 31 Wang Lin, Zhengfeng Yang, Xin Chen, Qingye Zhao, Xiangkun Li, Zhiming Liu, and Jifeng
 1074 He. Robustness Verification of Classification Deep Neural Networks via Linear Programming.
 1075 In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages
 1076 11410–11419, 2019. doi:10.1109/CVPR.2019.01168.
- 1077 32 Changliu Liu, Tomer Arnon, Christopher Lazarus, Christopher Strong, Clark Barrett, and
 1078 Mykel J. Kochenderfer. Algorithms for Verifying Deep Neural Networks. *Foundations*
 1079 *and Trends in Optimization*, 4(3-4):244–404, 2021. URL: [http://dx.doi.org/10.1561/](http://dx.doi.org/10.1561/24000000035)
 1080 [24000000035](http://dx.doi.org/10.1561/24000000035), doi:10.1561/24000000035.
- 1081 33 Liya Liu, Vincent Aravantinos, Osman Hasan, and Sofiène Tahar. On the Formal Analysis of
 1082 HMM Using Theorem Proving. In Stephan Merz and Jun Pang, editors, *Formal Methods and*
 1083 *Software Engineering*, pages 316–331, Cham, 2014. Springer International Publishing.
- 1084 34 mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM*
 1085 *SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages
 1086 367–381, 2020.
- 1087 35 Sean Meyn, Richard L. Tweedie, and Peter W. Glynn. *Markov Chains and Stochastic Stability*.
 1088 Cambridge Mathematical Library. Cambridge University Press, 2 edition, 2009.
- 1089 36 Charlie Murphy, Patrick Gray, and Gordon Stewart. Verified Perceptron Convergence Theorem.
 1090 In *Proceedings of the 1st ACM SIGPLAN International Workshop on Machine Learning and*
 1091 *Programming Languages*, pages 43–50, 2017.
- 1092 37 Michael Norrish and Konrad Slind. *The HOL System DESCRIPTION*. 3rd edition, 2022.
- 1093 38 Hiroyuki Okazaki, Yuichi Futa, and Yasunari Shidama. Formal Definition of Probability on
 1094 Finite and Discrete Sample Space for Proving Security of Cryptographic Systems Using Mizar.
 1095 *Artificial Intelligence Research*, 2, 2013. doi:10.5430/air.v2n4p37.
- 1096 39 Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan,
 1097 Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas
 1098 Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy,
 1099 Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. *PyTorch: An Imperative Style,*
 1100 *High-Performance Deep Learning Library*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- 1101 40 Christian P. Robert and George Casella. *Monte Carlo Statistical Methods*. Springer Texts in
 1102 Statistics. Springer, 1999.
- 1103 41 Daniel Selsam, Percy Liang, and David L. Dill. Developing Bug-Free Machine Learning
 1104 Systems with Formal Mathematics. In *Proceedings of the 34th International Conference on*
 1105 *Machine Learning - Volume 70, ICML’17*, page 3047–3056. JMLR.org, 2017.
- 1106 42 E. Seneta. *Non-negative Matrices and Markov Chains*. Springer, New York, 2006.
- 1107 43 Joseph Tassarotti and Robert Harper. A Separation Logic for Concurrent Randomized
 1108 Programs. *Proceedings of the ACM on Programming Languages*, 3(POPL):1–30, 2019.

- 1109 **44** Joseph Tassarotti, Koundinya Vajjha, Anindya Banerjee, and Jean-Baptiste Tristan. A Formal
1110 Proof of PAC Learnability for Decision Stumps. In *Proceedings of the 10th ACM SIGPLAN*
1111 *International Conference on Certified Programs and Proofs*, CPP 2021, page 5–17, New York,
1112 NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3437992.3439917.
- 1113 **45** René Thiemann. Stochastic matrices and the perron-frobenius theorem. *Archive of Formal*
1114 *Proofs*, November 2017. http://isa-afp.org/entries/Stochastic_Matrices.html, Formal
1115 proof development.
- 1116 **46** René Thiemann and Akihisa Yamada. Formalizing jordan normal forms in isabelle/hol.
1117 In *Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs*,
1118 CPP 2016, page 88–99, New York, NY, USA, 2016. Association for Computing Machinery.
1119 doi:10.1145/2854065.2854073.
- 1120 **47** Joseph Tooby-Smith. HepLean: Digitalising High Energy Physics. *Comput. Phys. Commun.*,
1121 308:109457, 2025. arXiv:2405.08863, doi:10.1016/j.cpc.2024.109457.
- 1122 **48** Hoang-Dung Tran, Stanley Bak, Weiming Xiang, and Taylor T. Johnson. Verification of
1123 Deep Convolutional Neural Networks Using Imagestars. In Shuvendu K. Lahiri and Chao
1124 Wang, editors, *Computer Aided Verification*, pages 18–42, Cham, 2020. Springer International
1125 Publishing.
- 1126 **49** Jean-Baptiste Tristan, Joseph Tassarotti, Koundinya Vajjha, Michael L. Wick, and Anindya
1127 Banerjee. Verification of ML Systems via Reparameterization, 2020. URL: <https://arxiv.org/abs/2007.06776>, arXiv:2007.06776.
- 1128 **50** Koundinya Vajjha, Avraham Shinnar, Barry Trager, Vasily Pestun, and Nathan Fulton.
1129 Certrl: formalizing convergence proofs for value and policy iteration in coq. In *Proceedings*
1130 *of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs*,
1131 CPP 2021, page 18–31, New York, NY, USA, 2021. Association for Computing Machinery.
1132 doi:10.1145/3437992.3439927.
- 1133 **51** Koundinya Vajjha, Barry Trager, Avraham Shinnar, and Vasily Pestun. Formalization of a
1134 Stochastic Approximation Theorem. In June Andronick and Leonardo de Moura, editors, *13th*
1135 *International Conference on Interactive Theorem Proving (ITP 2022)*, volume 237 of *Leibniz*
1136 *International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:18, Dagstuhl, Germany,
1137 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2022.31>, doi:10.4230/LIPIcs.ITP.2022.31.
- 1138 **52** Kexing Ying and Rémy Degenne. A Formalization of Doob’s Martingale Convergence Theorems
1139 in Mathlib. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified*
1140 *Programs and Proofs*, CPP 2023, page 334–347, New York, NY, USA, 2023. Association for
1141 Computing Machinery. doi:10.1145/3573105.3575675.
1142
1143